

我们在后面将介绍四种分析表的构造方法，它们是：

- (1) LR(0)表构造法：这种方法局限性很大，但它是建立一般LR分析表的基础；
- (2) SLR(1)表(即简单LR表)构造法：这种方法较易实现又极有使用价值；
- (3) LR(1)表(即规范LR表)构造法：这种表适用大多数上下文无关文法，但分析表体积庞大；
- (4) LALR表(即向前LR表)构造法：该表能力介于SLR(1)和LR(1)之间。

3.5.2 LR(0)分析表的构造

我们希望仅由一种只概括“历史”资料而不包含推测性“展望”材料的简单状态就能识别呈现在栈顶的某些句柄，而LR(0)项目集就是这样一种简单状态。

在讨论LR分析法时，需要定义一个重要概念，这就是文法规范句型的“活前缀”。字的前缀是指该字的任意首部，例如字abc的前缀有 ε 、a、ab或abc。所谓活前缀，是指规范句型的一个前缀，这种前缀不含句柄之后的任何符号。在LR分析工作过程中的任何时候，栈里的文法符号(自栈底而上) $X_1X_2\ldots X_m$ 应该构成活前缀，

把输入串的剩余部分匹配于其后即应成为规范句型(如果整个输入串确为一个句子的话)。因此，只要输入串的已扫描部分保持可归约成一个活前缀，就意味着所扫描的部分没有错误。

对于一个文法 $G[S]$ ，首先要构造一个NFA，它能识别 $G[S]$ 的所有活前缀。这个NFA的每个状态就是一个“项目”，文法 $G[S]$ 中每一个产生式的右部添加一个圆点，称为 $G[S]$ 的一个LR(0)项目(简称项目)。

例如，产生式 $A \rightarrow XYZ$ 对应四个项目：

$$A \rightarrow \cdot X Y Z$$

$$A \rightarrow X \cdot Y Z$$

$$A \rightarrow X Y \cdot Z$$

$$A \rightarrow X Y Z \cdot$$

但是产生式 $A \rightarrow \varepsilon$ 只对应一个项目 $A \rightarrow \cdot$ 。一个项目指明了在分析过程的某个时刻我们看到产生式的多大一部分。可以使用这些项目状态构造一个NFA用来识别文法的所有活前缀。

使用第二章讲述的子集方法，就能够把识别活前缀的NFA确定化，使之成为一个以项目集合为状态的DFA，这个DFA就是建立LR分析算法的基础。构成识别一个文法活前缀的DFA的项目集(状态)的全体称为这个文法的LR(0)项目集规范族。这个规范族提供了建立一类LR(0)和SLR(1)分析器的基础。注意，凡圆点在最右端的项目，如 $A \rightarrow \alpha \cdot$ ，称为一个“归约项目”；对文法的开始符号 S' 的归约项目(S' 的含义见下面的叙述)，如 $S' \rightarrow \alpha \cdot$ ，称为“接受”项目；形如 $A \rightarrow \alpha \cdot a \beta$ 的项目(其中 a 为终结符)，称为“移进”项目；形如 $A \rightarrow \alpha \cdot B \beta$ 的项目(其中 B 为非终结符)，称为“待约”项目。

1. LR(0)项目集规范族的构造

我们用第二章所引进的 $\varepsilon_CLOSURE$ (闭包)来构造一个文法 $G[S]$ 的LR(0)项目集规范族。为了使“接受”状态易于识别，总是将文法 $G[S]$ 进行拓广。假定文法 $G[S]$ 以 S 为开始符号，我们构造一个 $G'[S']$ ，它包含了整个 $G[S]$ 并引进了一个不出现在 $G[S]$ 中的非终结符 S' ，同时加进了一个新产生式 $S' \rightarrow S$ ，这个 S' 是 $G'[S']$ 的开始符号。称 $G'[S']$ 是 $G[S]$ 的拓广文法，并且会有一个仅含项目 $S' \rightarrow S \cdot$ 的状态，这就是惟一的“接受”态。

假定 I 是文法 $G'[S']$ 的任一项目集，则定义和构造 I 的闭包 $CLOSURE(I)$ 的方法是：

(1) I 的任何项目都属于 $CLOSURE(I)$ ；

(2) 若 $A \rightarrow \alpha \cdot B \beta$ 属于 $CLOSURE(I)$ ，那么对任何关于 B 的产生式 $B \rightarrow \gamma$ ，其项目 $B \rightarrow \cdot \gamma$ 也属于 $CLOSURE(I)$ (设 $A \rightarrow \alpha \cdot B \beta$ 的状态为 i ，则 i 到所有含 $B \rightarrow \cdot \gamma$ 的状态都有一条 ε 有向边，即此规则仍与第二章的 $\varepsilon_CLOSURE(I)$ 定义一样)；

(3) 重复执行上述(1)~(2)步直至 $CLOSURE(I)$ 不再增大为止。

在构造CLOSURE(I)时请注意一个重要的事实，那就是对任何非终结符B，若某个圆点在左边的项目 $B \rightarrow \cdot \gamma$ 进入到CLOSURE(I)，则B的所有其它圆点在左边的项目 $B \rightarrow \cdot \beta$ 也将进入同一个CLOSURE集。

此外，我们设函数GO为状态转换函数，GO(I,X)的第一个变元I是一个项目集，第二个变元X是一个文法符号(终结符或非终结符)，函数GO(I,X)定义为

$$GO(I,X)=CLOSURE(J)$$

其中, 如果 $A \rightarrow \alpha \cdot X \beta$ 属于 I , 则 $J = \{ \text{任何形如 } A \rightarrow \alpha X \cdot \beta \text{ 的项目} \}$ 。如果由 I 项目集发出的字符为 X 的有向边, 则到达的状态即为 $\text{CLOSURE}(J)$ (这也类同于第二章 $I_a = \epsilon_ \text{CLOSURE}(J)$ 的定义, 但这里相当于输入的字符是 X)。直观上说, 若 I 是对某个活前缀 γ 有效的项目集(状态), 则 $\text{GO}(I, X)$ 就是对 γX 有效的项目集(状态)。

通过函数CLOSURE和GO很容易构造一个文法 $G[S]$ 的拓广文法 $G'[S']$ 的LR(0)项目集规范族。如果已经求出了 I 的闭包CLOSURE(I)，则用状态转换函数GO可以求出由项目集 I 到另一项目集状态必须满足的字符(即转换图有向边上的字符)；然后，再求出有向边到达的状态所含的项目集，即用 $GO(I,X)=CLOSURE(J)$ 求出 J ，再对 J 求其闭包CLOSURE(J)，也就是有向边到达状态所含的项目集。以此类推，最终构造出拓广文法 $G'[S']$ 的LR(0)项目集规范族。

4.6.3、LR(0) 分析

LR(0) 文法能力最弱，但是是其它文法的理论基础

- 存在DFA 识别活前缀
- 如何构造识别活前缀的DFA (构造LR(0)项目集规范族, 一个项目集对应DFA的一个状态)
- LR(0)分析表的构造

LR(0) 项目

首先扩充文法初始符号 $S' \rightarrow S$

(为使初始符不出现在任何产生式的右部)

- 0) $S' \rightarrow S$
- 1) $S \rightarrow B B$
- 2) $B \rightarrow a B$
- 3) $B \rightarrow b$

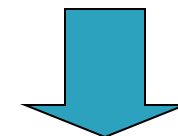
- 产生式右部有 n 个符号, 则有 $n+1$ 个位置
- LR(0)项目是在产生式右部某个位置上标分割圆点
- (x, y) 表示 x 号产生式、 y 号位置的LR(0)项目

LR(0) 项目:

- $(0, 0) \quad S' \rightarrow \cdot S$
- $(0, 1) \quad S' \rightarrow S \cdot$
- $(1, 0) \quad S \rightarrow \cdot BB$
- $(1, 1) \quad S \rightarrow B \cdot B$
- $(1, 2) \quad S \rightarrow BB \cdot$
- $(2, 0) \quad B \rightarrow \cdot a B$
- $(2, 1) \quad B \rightarrow a \cdot B$
- $(2, 2) \quad B \rightarrow a B \cdot$
- $(3, 0) \quad B \rightarrow \cdot b$
- $(3, 1) \quad B \rightarrow b \cdot$

求项目集（状态）

➤ 求核心项目的闭包 $\text{closure}(I)$



$J := I;$

repeat

$J = J \cup \{ B \rightarrow \cdot \eta \mid A \rightarrow \alpha \cdot B \beta \in J, \alpha, \beta, \eta \in V^*, B \in V_N \}$

until J 不再扩大

0)	$S' \rightarrow S$
1)	$S \rightarrow B B$
2)	$B \rightarrow a B$
3)	$B \rightarrow b$

例:

$\text{closure}((0, 0))$

$= \text{closure}(S' \rightarrow \cdot S)$

$= (0, 0), (1, 0), (2, 0), (3, 0)$

$S \rightarrow \cdot BB$

$B \rightarrow \cdot aB$

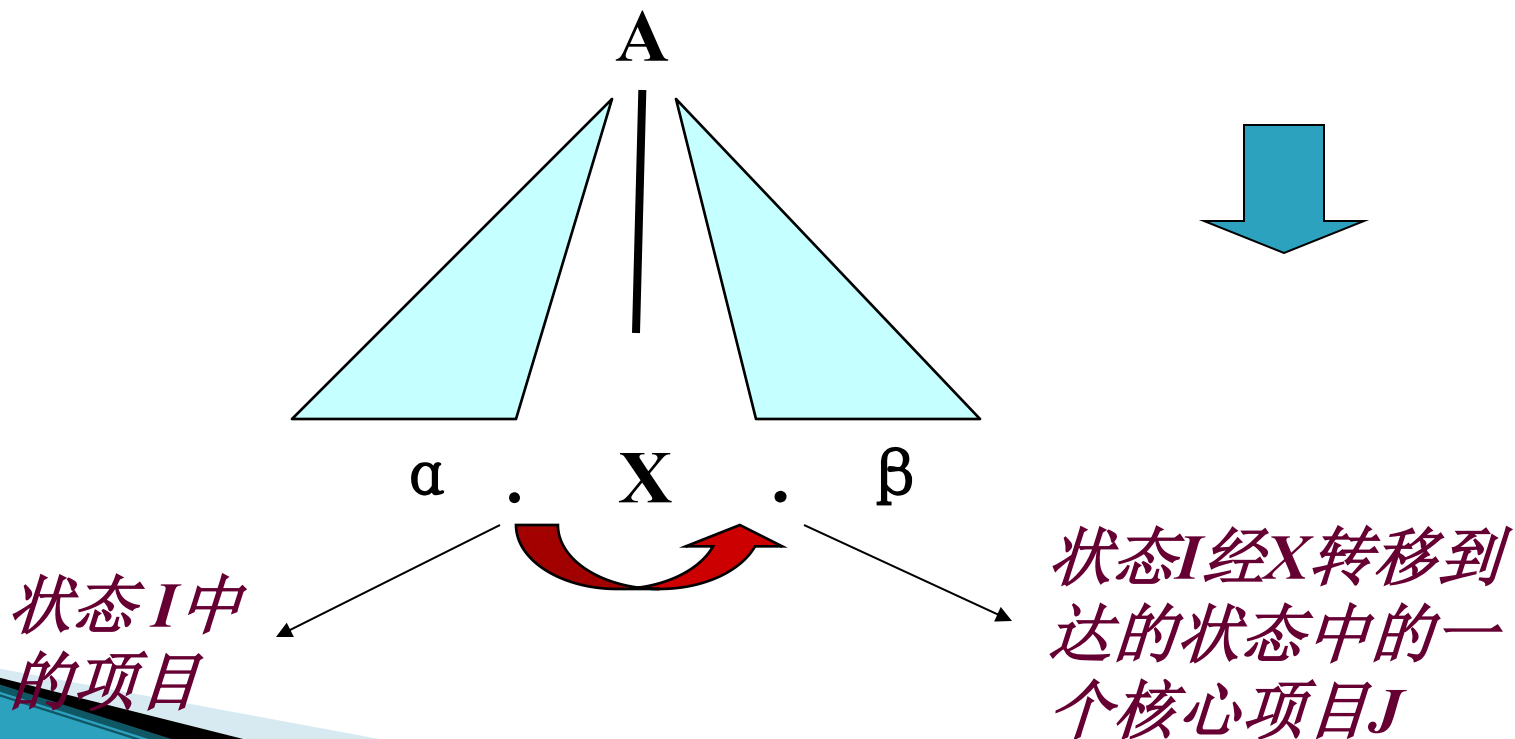
$B \rightarrow \cdot b$

- 状态转移的计算:

确定在某状态（项目集）遇到一个文法符号后的目标转移状态（项目集）

$$\text{go}(I, X) = \text{closure}(J)$$

其中 $J = \{ A \rightarrow \alpha X \beta \mid A \rightarrow \alpha \cdot X \beta \in I, X \in V \}$



- 计算LR(0)项目集规范族:

begin

$I := \text{closure}(\{ S' \rightarrow .S \}); \quad // \text{closure}(0, 0)$

$C := \{ I \};$

repeat

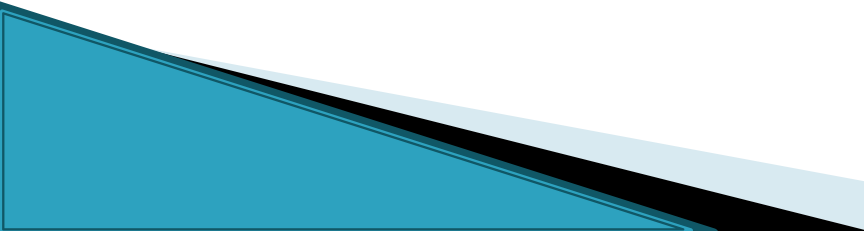
for C 中的每一状态 I 和每一文法符号 X

if $\text{go}(I, X)$ 非空且 $\text{go}(I, X)$ 不在 C 中

then 将状态 $\text{go}(I, X)$ 加入到 C 中

until 不出现新的状态

end.



用 $\varepsilon_CLOSURE$ 办法构造文法 $G'[S']$ 的 LR(0) 项目集规范族如下:

$I_0: S' \rightarrow \cdot S$	$I_1: S' \rightarrow S \cdot$	$I_3: B \rightarrow a \cdot B$	$I_5: S \rightarrow BB \cdot$
$S \rightarrow \cdot BB$	$I_2: S \rightarrow B \cdot B$	$B \rightarrow \cdot aB$	$I_6: B \rightarrow aB \cdot$
$B \rightarrow \cdot aB$	$B \rightarrow \cdot aB$	$B \rightarrow \cdot b$	
$B \rightarrow \cdot b$	$B \rightarrow \cdot b$	$I_4: B \rightarrow b \cdot$	

根据状态转换函数 GO 构造出文法 $G'[S']$ 的 DFA, 如图 3-22 所示。

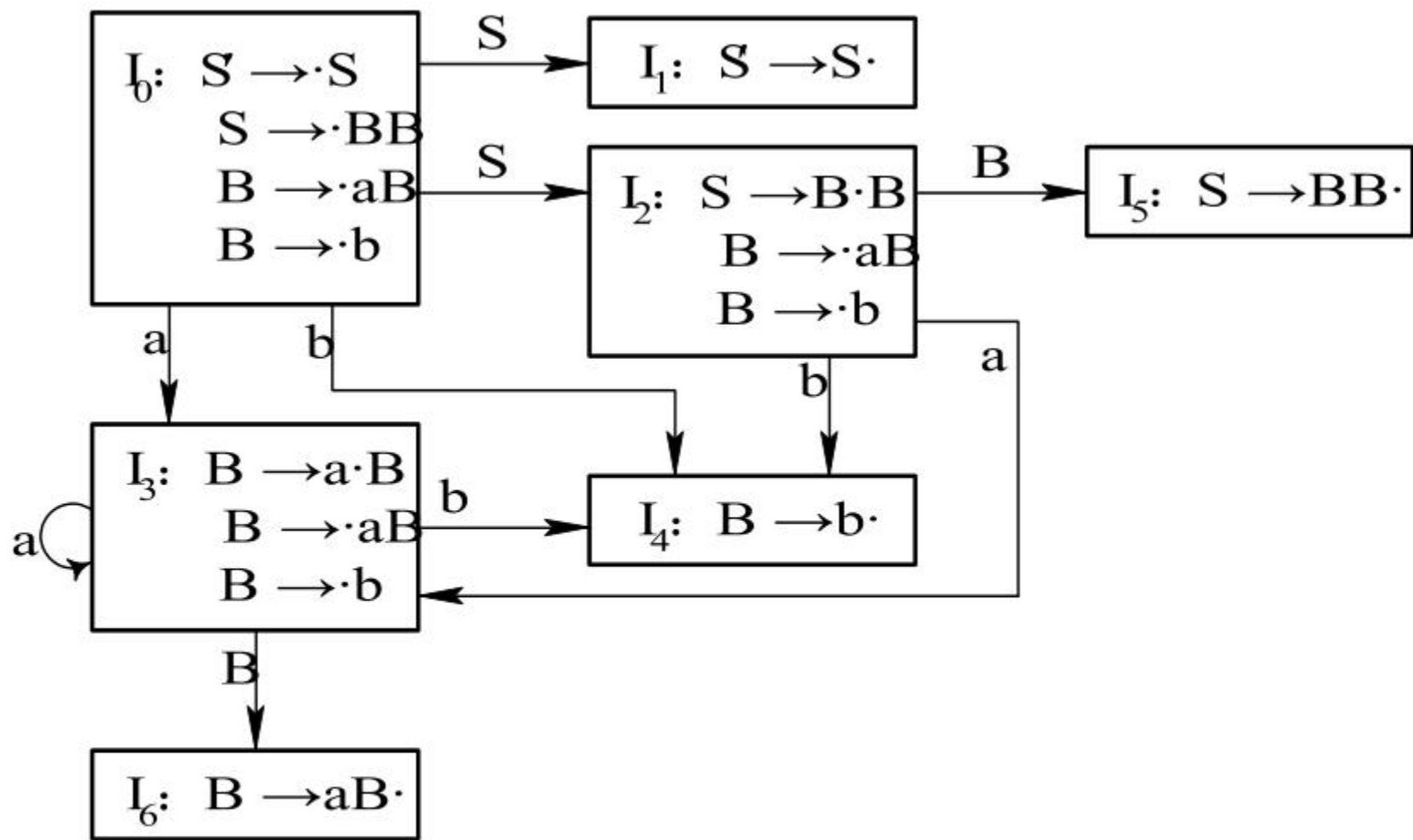


图3-22 例3.16文法 $G'[S']$ 的DFA(即LR(0)项目集和GO函数)

表3.15 例3.16的LR(0)分析表

状态	ACTION			GOTO	
	a	b	#	S	B
0	s ₃	s ₄		1	2
1			acc		
2	s ₃	s ₄			5
3	s ₃	s ₄			6
4	r ₃	r ₃	r ₃		
5	r ₁	r ₁	r ₁		
6	r ₂	r ₂	r ₂		

例1: 表达式拓展文法G[E]

- 0) $E' \rightarrow 0 \quad E \ 1$
- 1) $E \rightarrow 0, 4 \quad E \ 1, 8 \ + \ 6 \quad T9$
- 2) $E \rightarrow 0, 4 \quad T \ 2$
- 3) $T \rightarrow 0, 4, 6 \quad T \ 2, 9 \ * \ 7 \quad F10$
- 4) $T \rightarrow 0, 4, 6 \quad F \ 3$
- 5) $F \rightarrow 0, 4, 6, 7(\ 4 \quad E8 \)11$
- 6) $F \rightarrow 0, 4, 6, 7^{id} \ 5$

- | | |
|--------------------------|--------------------------|
| 0) $E' \rightarrow E$ | 3) $T \rightarrow T * F$ |
| 1) $E \rightarrow E + T$ | 4) $T \rightarrow F$ |
| 2) $E \rightarrow T$ | 5) $F \rightarrow (E)$ |
| | 6) $F \rightarrow id$ |

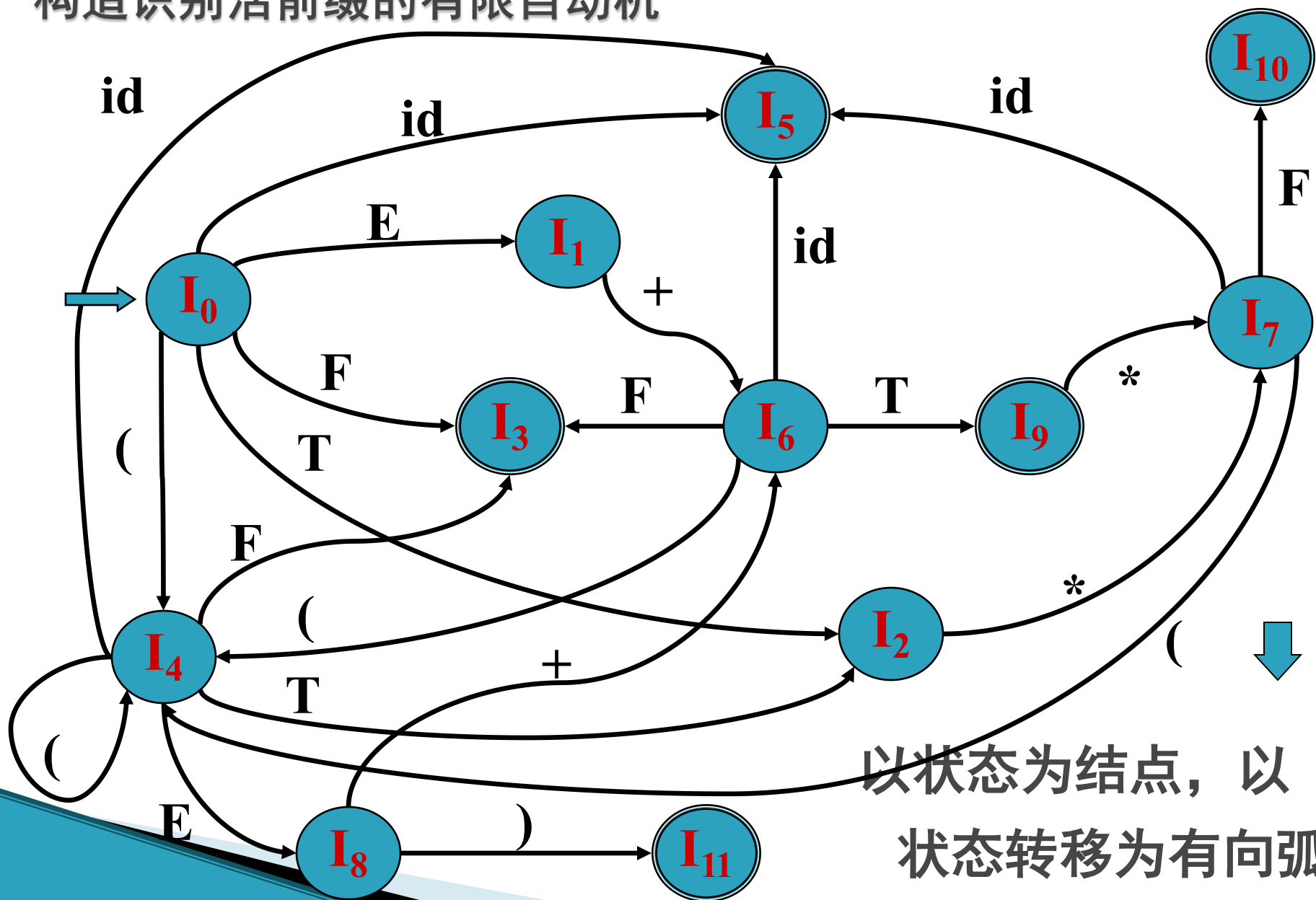


Go(I, X)	状态I	项目核心J	计算closure(J)后添加的项目
	0	(0, 0)	(1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
go(0, E)	1	(0, 1) (1, 1)	
go(0, T)	2	(2, 1) (3, 1)	
go(0, F)	3	(4, 1)	
go(0, ())	4	(5, 1)	(1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
go(0, id)	5	(6, 1)	
go(1, +)	6	(1, 2)	(3, 0) (4, 0) (5, 0) (6, 0)
go(2, *)	7	(3, 2)	(5, 0) (6, 0)
go(4, E)	8	(5, 2) (1, 1)	
go(4, T)	2	(2, 1) (3, 1)	

go(4, F)	3	(4, 1)	
go(4, ())	4	(5, 1)	(1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
go(4, id)	5	(6, 1)	
go(6, T)	9	(1, 3) (3, 1)	
go(6, F)	3	(4, 1)	
go(6, ())	4	(5, 1)	(1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
go(6, id)	5	(6, 1)	
go(7, F)	10	(3, 3)	
go(7, ())	4	(5, 1)	(1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
go(7, id)	5	(6, 1)	
go(8,))	11	(5, 3)	
go(8, +)	6	(1, 2)	(3, 0) (4, 0) (5, 0) (6, 0)
go(9, *)	7	(3, 2)	(5, 0) (6, 0)

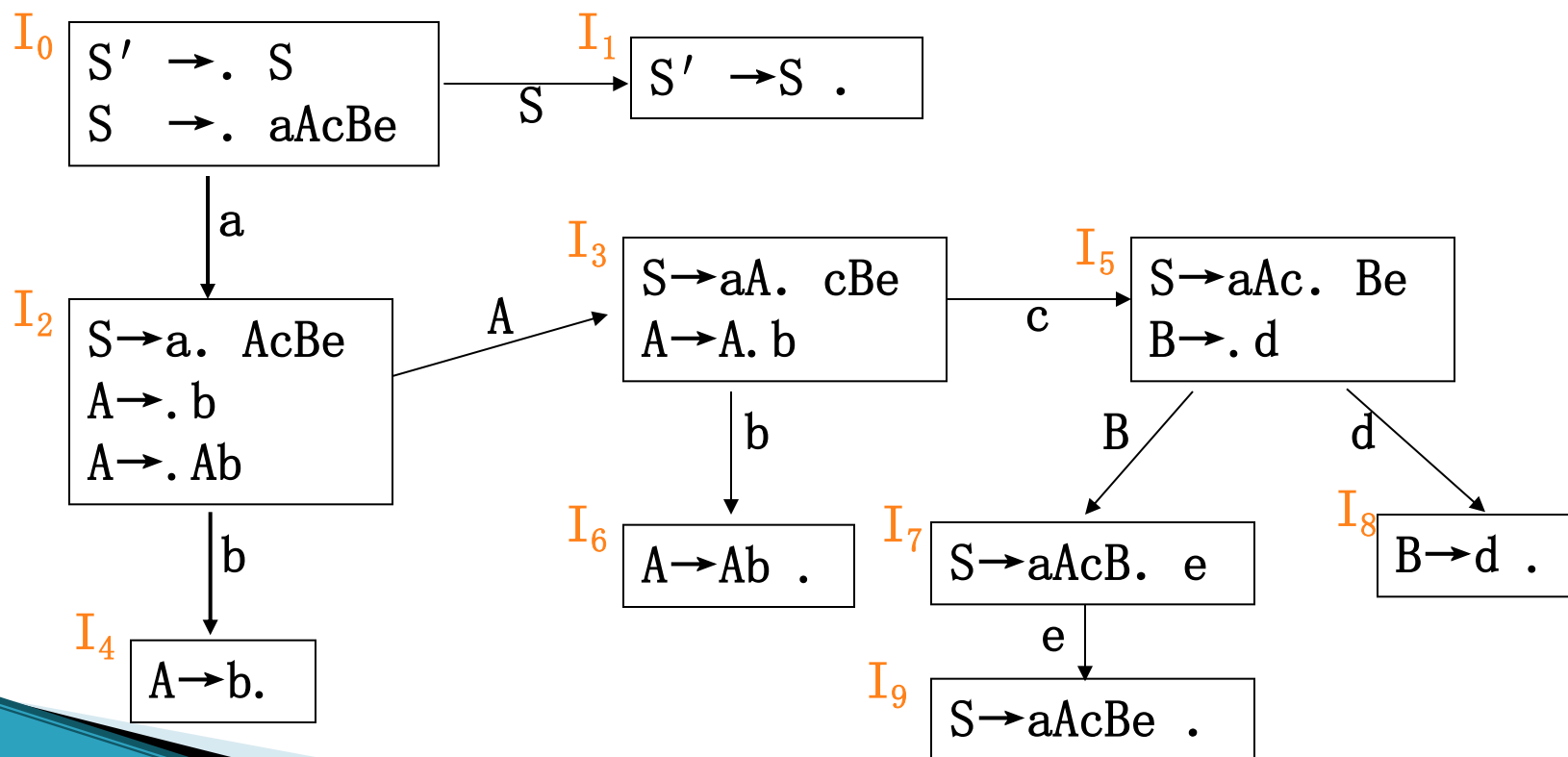
至此，所有状态转移的可能都被考虑到了，不可能再产生新的状态。

构造识别活前缀的有限自动机



例2: $G[S]$ 的扩展文法:

- 0) $S' \rightarrow S$
- 1) $S \rightarrow a A c B e$
- 2) $A \rightarrow b$
- 3) $A \rightarrow Ab$
- 4) $B \rightarrow d$



LR(0) 分析中构造Action和Goto表的算法:

1. 构造扩展文法 G' 的状态集合 $\{ I_0, I_1, \dots, I_n \}$
2. action为一维表, 状态 I_i 的分析动作为:
 - a) if $A \rightarrow \alpha . a \beta \in I_i \quad a \in V_T$ then
将 'S' 置入 action[i] //做移进动作
 - b) if $A \rightarrow \alpha . \in I_i$ then //在识别 α 之后, 应做归约
将 'rj' 置入action[i] // $A \rightarrow \alpha$ 为文法的第j个产生式
 - c) if $S' \rightarrow S. \in I_i$ then
将 'acc' 置入 action[i] //接受, 分析成功
3. goto为二维表, 行为状态, 列为文法符号X。
if $go(I_i, X) = I_j$ then 将j置入goto[i, X] //设置状态转移
4. 所有空格置 error
5. 以 $S' \rightarrow .S$ (即项目(0, 0)) 所在的状态为初态

例1的ACTION表和GOTO表:

	ACTION 	GOTO 							
		+	*	(	)	id	E	T	F
0	S			4		5	1	2	3
1	acc、S	6							
2	r2、S		7						
3	r4								
4	S			4		5	1	2	3
5	r6								
6	S			4		5		9	3
7	S			4		5			10
8	S	6			11				
9	S、r1		7						
10	r3								
11	r5								



	ACT ION	GOTO								
		a	c	e	b	d		S	A	B
0	S	2						1		
1	acc									
2	S				4				3	
3	S		5		6					
4	r ₂									
5	S					8				7
6	r ₃									
7	S			9						
8	r ₄									
9	r ₁									

对输入串 **abbcde#** 的分析过程

<u>Step</u>	<u>states.</u>	<u>Syms.</u>	<u>The rest of input</u>	<u>action</u>	<u>goto</u>
1	0	#	abbcde#	s2	
2	02	#a	bbcde#	s4	
3	024	#ab	bcde#	r2	3
4	023	#aA	bcde#	s6	
5	0236	#aAb	cde#	r3	3
6	023	#aA	cde#	s5	
7	0235	#aAc	de#	s8	
8	02358	#aAcd	e#	r4	7
9	02357	#aAcB	e#	s9	
10	023579	#aAcBe	#	r1	1
11	01	#S	#	acc	

对输入串 **abbce#** 的分析过程

<u>Step</u>	<u>states.</u>	<u>Syms.</u>	<u>The rest of input</u>	<u>action</u>	<u>goto</u>
1	0	#	abbce#	s2	
2	02	#a	bbce#	s4	
3	0 24	#a b	bce#	r2	3
4	023	#aA	bce#	s6	
5	0 236	#aA b	ce#	r3	3
6	023	#aA	ce#	s5	
7	0235	#aAc	e#	出错	

说明abbce#不是 文法 $G[S]$ 的句子

表中入口含多重定义的情况：

文法有移进-归约冲突或归约-归约冲突的时候。

按上述算法构造的含有ACTION和GOTO两部分的分析表，如果每个入口不含多重定义，则称它为文法G的LR(0)表。具有LR(0)表的文法G称为LR(0)文法。

LR(0)文法是无二义的。



LR(0)分析算法

- ▶ 置ip指向输入串w的第一个符号，初始状态入状态栈，#入符号栈
 - 令St为栈顶状态，a是ip指向的符号
 - LOOP
- begin
 - if $S \in \text{ACTION}[St]$
 - then begin PUSH GOTO(St, a), a(进栈——状态栈和符号栈)
 - ip 前进(指向下一输入符号)
 - end
 - else if $r_j \in \text{ACTION}[St]$ (第j条产生式为 $A \rightarrow \beta$)
 - then begin 两个栈各 pop $|\beta|$ 项
 - 假设当前状态栈顶状态为 St' ，则
 - push GOTO[St' , A]和A
 - end
 - else if $acc \in \text{ACTION}[St]$
 - then return (成功)
 - else error
- end. LOOP

例3.17 8086/8088汇编语言对操作数域的检查可以用LR分析表实现。设m代表存储器，r代表寄存器，i代表立即数；并且为了简单起见，省去了关于m、r和i的产生式，暂且认为m、r、i为终结符，则操作数域P的文法G[P]为

$$G[P]: P \rightarrow m, r \mid m, i \mid r, r \mid r, i \mid r, m$$

试构造能够正确识别操作数域的LR分析表。

[解答] (1) 将文法 $G[S]$ 拓广为文法 $G'[S']$:

$G'[S']$: (0) $S' \rightarrow P$

(1) $P \rightarrow m, r$

(2) $P \rightarrow m, i$

(3) $P \rightarrow r, r$

(4) $P \rightarrow r, i$

(5) $P \rightarrow r, m$

(2) 列出LR(0)的所有项目：

1. $S' \rightarrow \cdot P$

9. $P \rightarrow m, \cdot i$

17. $P \rightarrow r, \cdot i$

2. $S' \rightarrow P \cdot$

10. $P \rightarrow m, i \cdot$

18. $P \rightarrow r, i \cdot$

3. $P \rightarrow \cdot m, r$

11. $P \rightarrow \cdot r, r$

19. $P \rightarrow \cdot r, m$

4. $P \rightarrow m \cdot, r$

12. $P \rightarrow r \cdot, r$

20. $P \rightarrow r \cdot, m$

5. $P \rightarrow m, \cdot r$

13. $P \rightarrow r, \cdot r$

21. $P \rightarrow r, \cdot m$

6. $P \rightarrow m, r \cdot$

14. $P \rightarrow r, r \cdot$

22. $P \rightarrow r, m \cdot$

7. $P \rightarrow \cdot m, i$

15. $P \rightarrow \cdot r, i$

8. $P \rightarrow m \cdot, i$

16. $P \rightarrow r \cdot, i$

(3) 用 $\varepsilon_CLOSURE$ (闭包)方法得到文法 $G'[S']$ 的LR(0)项目集规范族如下:

$I_0: S' \rightarrow \cdot P$	$I_1: P \rightarrow m \cdot, r$	$I_3: P \rightarrow r, \cdot r$	$I_6: P \rightarrow m, i \cdot$
$P \rightarrow \cdot m, r$	$P \rightarrow m \cdot, i$	$P \rightarrow r, \cdot i$	$I_7: P \rightarrow r, r \cdot$
$P \rightarrow \cdot m, i$	$I_2: P \rightarrow r \cdot, r$	$P \rightarrow r, \cdot m$	$I_8: P \rightarrow r, i \cdot$
$P \rightarrow \cdot r, r$	$P \rightarrow r \cdot, i$	$I_4: P \rightarrow m, \cdot r$	$I_9: P \rightarrow r, m \cdot$
$P \rightarrow \cdot r, i$	$P \rightarrow r \cdot, m$	$P \rightarrow m, \cdot i$	$I_A: S' \rightarrow P \cdot$
$P \rightarrow \cdot r, m$	$I_5: P \rightarrow m, r \cdot$		

(4) 根据状态转换函数GO构造出文法 $G'[S']$ 的DFA, 如图3-23所示。

(5) 构造LR(0)分析表如表3.16所示。

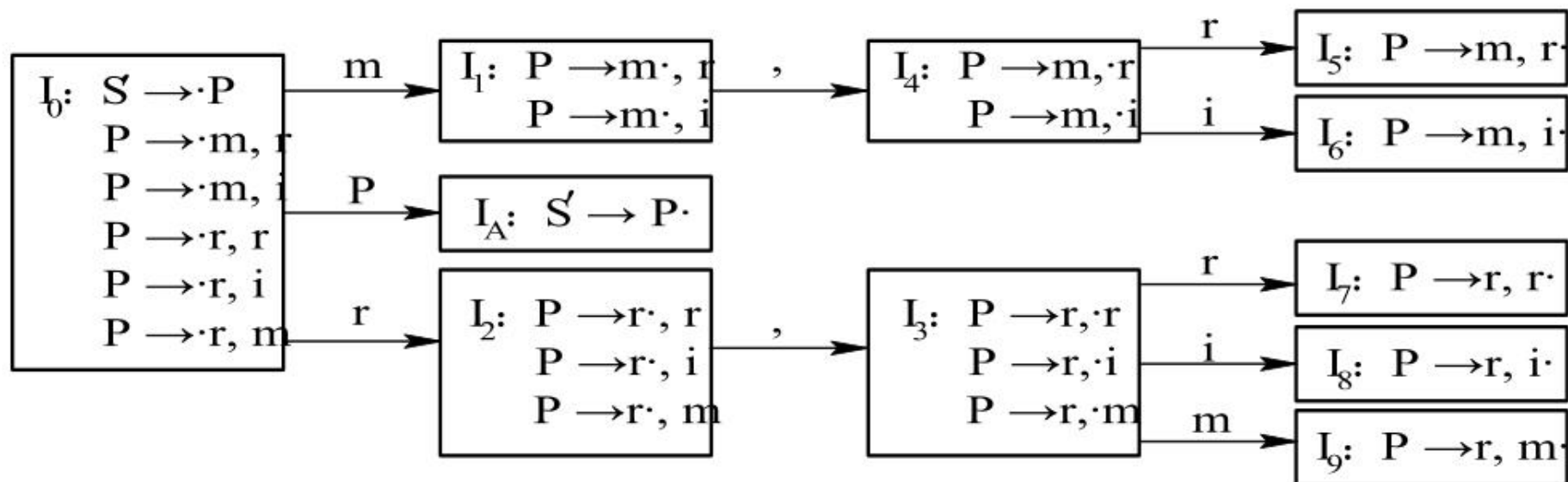


图3-23 例3.17文法 $G'[S']$ 的DFA

表3.16 例3.17的LR(0)分析表

状态	ACTION					GOTO
	m	r	i	,	#	
0	s ₁	s ₂				A
1				s ₄		
2				s ₃		
3	s ₉	s ₇	s ₈			
4		s ₅	s ₆			
5	r ₁	r ₁	r ₁	r ₁	r ₁	
6	r ₂	r ₂	r ₂	r ₂	r ₂	
7	r ₃	r ₃	r ₃	r ₃	r ₃	
8	r ₄	r ₄	r ₄	r ₄	r ₄	
9	r ₅	r ₅	r ₅	r ₅	r ₅	
A					acc	

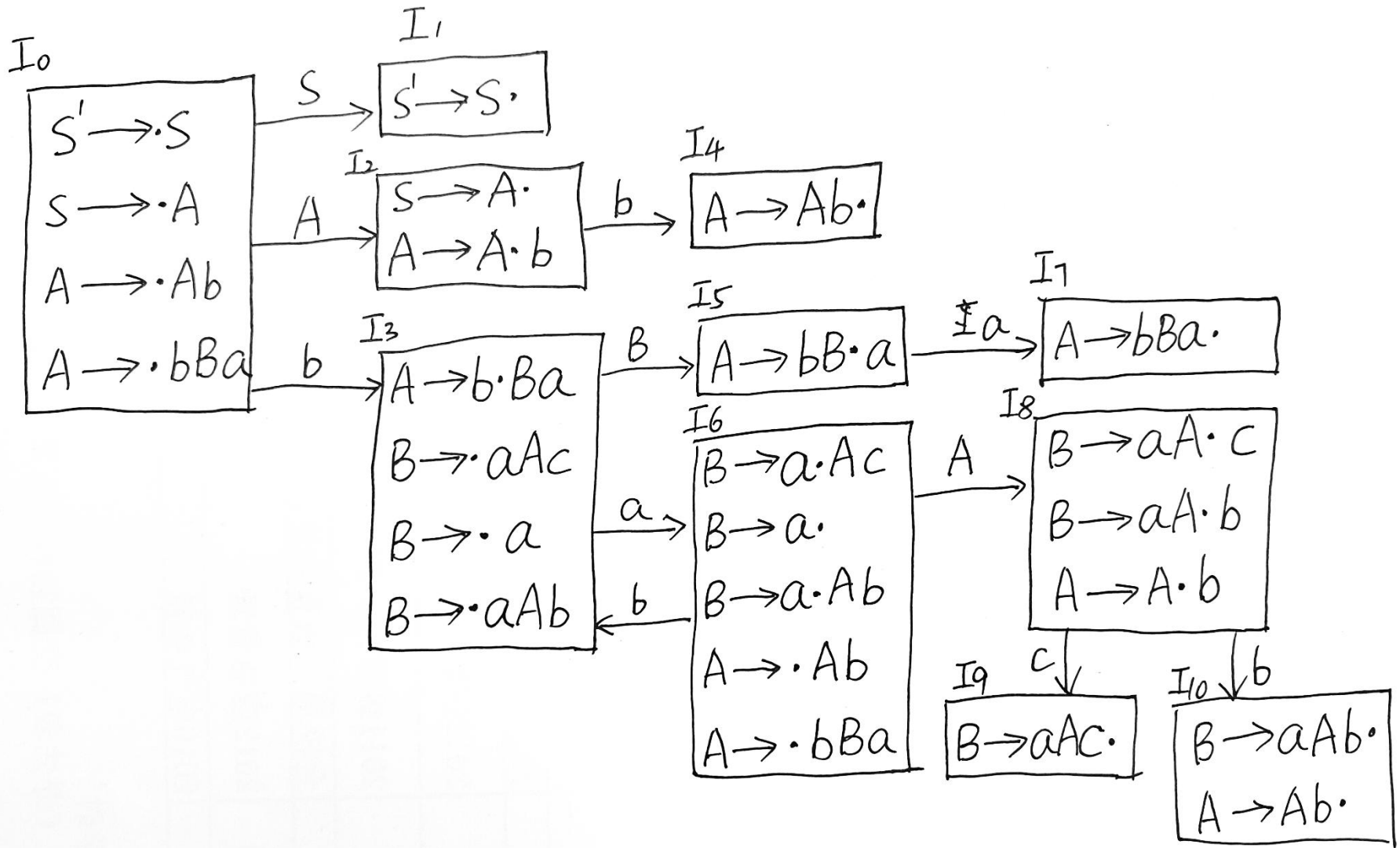
练习：文法G[S]

$S \rightarrow A$

$A \rightarrow Ab \mid bBa$

$B \rightarrow aAc \mid a \mid aAb$

- (1) 扩展文法，构造LR(0)项目集规范族和识别其活前缀的有限自动机
- (2) 构造LR(0)分析表（ACTION及GOTO表）
- (3) 该文法是否LR(0)文法？



		Action				Goto		
		a	b	c	#	S	A	B
S	0		3			1	2	
acc	1							
S, r1	2		4					
S	3	6						5
r2	4							
S	5	7						
S, r5	6		3				8	
r3	7							
S	8		10	9				
r4	9							
r2, r6	10							